

Preparation of Papers for IEEE ACCESS

FIRST A. AUTHOR¹, (Fellow, IEEE), SECOND B. AUTHOR², AND THIRD C. AUTHOR, Jr.³, (Member, IEEE)

¹National Institute of Standards and Technology, Boulder, CO 80305 USA (e-mail: author@boulder.nist.gov)

²Department of Physics, Colorado State University, Fort Collins, CO 80523 USA (e-mail: author@lamar.colostate.edu)

³Electrical Engineering Department, University of Colorado, Boulder, CO 80309 USA

Corresponding author: First A. Author (e-mail: author@boulder.nist.gov).

This paragraph of the first footnote will contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456."

ABSTRACT Current development of microservice architectures heavily relies on API gateways and high-performance RPC frameworks like gRPC, the implementation details of the inter-service communication layer are often abstracted away in system design. Our paper empirically investigates the performance degradation caused by application-layer communication bottlenecks, with particular focus on gRPC connection management. We present a mathematical queueing model and algorithmic formalization to quantify the overhead of the connection-per-request (CPR) compared to a persistent channel reuse (PCR). Our evaluation results in Kubernetes environment tests show that converting CPR to PCR models can significantly reduce median latency by 97%—from over 7,000 ms to 150 ms under peak load. This change significantly eliminates the performance bottleneck that caused an increase in end-to-end user experience latency time.

INDEX TERMS Cloud-Native, Connection Per Request, gRPC API, High Performance Computing, Kubernetes, Microservices, Persistent Channel Reuse

I. INTRODUCTION

THE transition to cloud-native microservices has established API gateways and high-performance RPC frameworks like gRPC as standard infrastructure. In this architecture, the API gateway functions as the singular entry point for external traffic, acting primarily as a reverse proxy and protocol transcoder. While external clients typically communicate using standard HTTP/1.1 and JSON payloads—often following RESTful paradigms as discussed in [1]—internal microservices increasingly rely on gRPC over HTTP/2 for high-performance communication [2]. Unlike the stateless nature of traditional REST, gRPC multiplexes multiple requests over a single persistent HTTP/2 connection. However, the implementation details of this inter-service communication layer are often abstracted away by frameworks. As demonstrated by Zhu et al. [3], significant infrastructural overheads remain hidden within service mesh sidecars; similarly, we observe that undocumented performance penalties exist within the application layer of API gateways when protocol translations are mismanaged.

The specific issue addressed in this paper is the performance degradation caused by the "connection-per-request" (CPR) anti-pattern within these gateways. Previous studies have shown how TCP and TLS handshake delays

reduce throughput in distributed systems [4]. However, applying legacy REST-based connection management to gRPC environments remains a pervasive flaw. While recent work on high-throughput serialization formats like Bebo highlights how CPU branch mispredictions and framing overheads bottleneck RPC runtimes [5], the fundamental overhead of connection instantiation at the gateway level is frequently overlooked. We define gRPC connection churn as the continuous cycle of establishing and tearing down complex network state (TCP handshakes and HTTP/2 protocol negotiations) for every individual application-layer request, rather than multiplexing traffic. Establishing a new connection per request is an $O(N)$ operation, where N represents the number of incoming client requests. This directly conflicts with the $O(1)$ multiplexed design of HTTP/2, forcing gateway worker threads to spend excessive CPU cycles on network context switching. We explore this vulnerability using a standard Python WSGI stack [6] (Flask and FastAPI) as a representative testbed, as its synchronous worker model clearly exposes the severe latency penalties of blocking network I/O.

While the research community has modeled microservice performance extensively, the specific intersection of application-layer connection churn and queueing network

saturation remains empirically unquantified. Recent works evaluate microservice resilience using stochastic Petri Nets [7] or optimize multi-service resource provisioning using stochastic execution models [8]. However, evaluating dynamic latency under load requires integrating these approaches with queueing theory [9]. We argue that implementation-level bottlenecks, specifically gRPC connection churn, dynamically inflate the service time (S) of the system, degrading the effective service rate (μ). This churn converts what should be a constant-time RPC execution into a highly variable, load-dependent event. Unmanaged connection churn destabilizes base duration estimates, forcing premature queue saturation long before physical CPU or memory thresholds are reached.

Therefore, this paper demonstrates how converting the CPR model into a Persistent Channel Reuse (PCR) model, as illustrated in Fig. 1, eliminates this bottleneck by refactoring the gateway code to initialize a global, thread-safe gRPC channel at startup. To validate this approach, the key contributions of this paper are as follows:

- We formalize and model the request patterns and application-layer execution logic for both the CPR and refactored PCR models.
- We introduce a queueing-theoretic model to explain non-linear latency spikes under high load, establishing a direct link between application-layer connection churn and queue saturation, thereby yielding a proposed controller that continuously senses for any change of catastrophic bottleneck to the frontend application.
- We benchmark these models within a production Kubernetes cluster using controlled and isolated environment, ensuring the gateway of both frontend and backend business logic variance is independently evaluated, to validate theoretical queueing projections against empirical execution runs.

II. RELATED WORK

A. TRANSPORT PROTOCOLS AND COMMUNICATION OVERHEAD IN MICROSERVICES

Microservice architectures increasingly rely on high-performance Remote Procedure Call (RPC) frameworks, such as gRPC, to facilitate low-latency inter-service communication over HTTP/2. Recent work reports that because gRPC uses HTTP/2 and Protocol Buffers, it can cut latency by about 30% compared with REST for inter-service communication [10]. That kind of gain matters most when applications make many small synchronous calls, since serialization cost, request framing, and connection handling are paid repeatedly across a request path.

While HTTP/2 multiplexing significantly reduces connection overhead compared to HTTP/1.1 REST interfaces, system performance remains highly sensitive to underlying transport-layer connection management. Existing benchmark studies demonstrate gRPC's superiority in execution time and resource utilization. Taha et al. point out that proactive

performance prediction in microservice systems struggles with dynamic dependencies and multiplexing, especially because long and changing call chains add delay [11]. When one service request triggers many downstream calls, queueing and contention on shared channels can make tail latency less predictable even if average latency stays low.

However, these studies predominantly assume persistent channel reuse (PCR) under steady-state workloads. Several aspects significantly influence and characterize the causes of increased tail latency and non-linear throughput degradation in cloud continuum environments—including ephemeral functions, rapid auto-scaling, connection-per-request (CPR) patterns, and connection churn—which frequently lead to the exhaustion of the OS TCP listen backlog. Biondani et al. found that gRPC performance degrades for large file transfers, and they attribute this to HTTP/2 transport behavior: over a 45 ms RTT path, the bandwidth-delay product interacts badly with HTTP/2 application-layer flow control, so throughput is throttled while senders wait for WINDOW_UPDATE frames [12]. This is important for inter-service design because it shows that HTTP/2 multiplexing helps most for many small concurrent exchanges, not for every traffic type.

B. CONTROL LOOPS AND TRAFFIC MANAGEMENT

The use of autonomic computing paradigms based on the Monitor-Analyze-Plan-Execute with Knowledge (MAPE-K) feedback loop has become common practice to address performance issues in cloud-native settings [13]. Traditional implementations integrate autonomic managers into the service mesh or API gateway to handle dynamic autoscaling, circuit breaking, anomaly detection, and traffic activity [8]. A recent article introduces machine learning and agentic control units to predict microservice anomalies [14]. Nevertheless, these approaches introduce non-negligible telemetry aggregation and decision latencies, making them less effective in responding to immediate network socket saturation [15]. Furthermore, standard load-balancing techniques used in gRPC, such as client-side round-robin or fixed connection age limits, do not effectively address connection starvation problems when runtime conditions suddenly shift between CPR and PCR operational modes [16].

C. POSITIONING AND NOVEL CONTRIBUTIONS

In contrast to existing static performance evaluations or resource-intensive AI-driven controller techniques, our approach introduces a lightweight, mathematically based autonomic controller tailored specifically for gRPC connection management. By clearly establishing the transition boundary between CPR and PCR paradigms through queueing dynamics, the proposed controller achieves sensing and actuation results within milliseconds. It reduces monitoring overhead in the baseline during normal operation, efficiently minimizes end-to-end tail latency (p_{99}) across

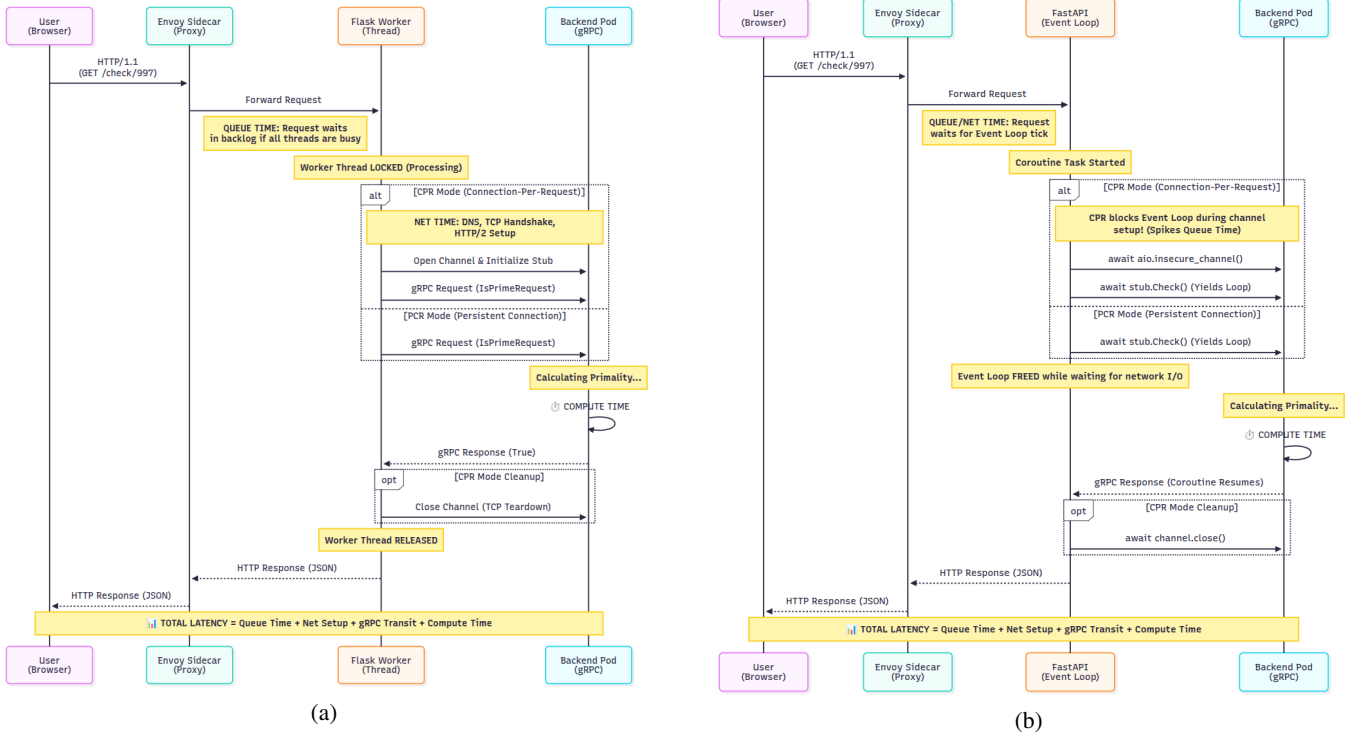


FIGURE 1. Sequence diagrams comparing synchronous and asynchronous communication models in cloud-native applications across Connection-Per-Request (CPR) and Persistent Connection Reuse (PCR): (a) Synchronous CPR/PCR (Flask/Gunicorn); (b) Asynchronous CPR/PCR (Flask/Uvicorn).

various load categories, and prevents throughput collapse during critical TCP backlog saturation.

III. MATHEMATICAL MODELING OF GRPC CONNECTION MANAGEMENT

In order to comprehend the communication bottleneck, it is essential to break down the entire lifecycle of a request as it progresses through the API gateway. This analysis will help us identify the key stages and factors contributing to the bottleneck in the communication process. We model this environment by evaluating two distinct application gateway architectures: a synchronous model [17] (e.g., Flask thread pools) and an asynchronous model [18] (e.g., FastAPI event loops).

Let T_{total} be the total end-to-end latency experienced by the client. We define T_{total} as:

$$T_{total} = T_{http} + W_{queue} + S_{gateway} + T_{network} \quad (1)$$

Where:

T_{http} : ingress HTTP processing time.

W_{queue} : queueing delay at the gateway's worker processes.

$S_{gateway}$: service time required by the gateway to establish and execute the downstream gRPC call.

$T_{network}$: physical network propagation delay and backend computation time (baseline constant $c \approx 3$ ms).

To isolate the backend processing efficiency from the communication overhead, $T_{network}$ is further decomposed

into pure server compute time ($T_{compute}$) and the combined network transit and backend queueing delay (T_{net_queue}):

$$T_{network} = T_{compute} + T_{net_queue} \quad (2)$$

During isolated backend service evaluation, these variables are extracted by aggregating raw container-level telemetry across all active backend pods. The pure compute time is derived from the cumulative execution time ($\sum T_{exec}$) and the total request count ($\sum N_{req}$):

$$T_{compute} = \left(\frac{\sum T_{exec}}{\sum N_{req}} \right) \times 1000 \quad (\text{ms}) \quad (3)$$

Subsequently, the network and internal queueing delay is isolated by subtracting the pure compute time from the measured average response latency (T_{avg}):

$$T_{net_queue} = T_{avg} - T_{compute} \quad (\text{ms}) \quad (4)$$

This decomposition proves that pure computation ($T_{compute}$) remains a fractional baseline constant (≈ 0.002 ms in our test environment), isolating gRPC connection handling, payload serialization, and queue saturation as the primary variables. Therefore, the critical variable in our end-to-end study is $S_{gateway}$. We model this variable under two distinct operational paradigms, representing the system state before and after the proposed controller's intervention.

A. PARADIGM 1: CONNECTION-PER-REQUEST (CPR)

In the Connection-Per-Request (CPR) anti-pattern, as shown in the CPR section of Fig. 1, the application gateway initializes a new gRPC channel for every incoming request. The service time S_{cpr} is defined as:

$$S_{cpr} = t_{tcp} + t_{http2} + t_{rpc} + t_{teardown} \quad (5)$$

Where t_{tcp} is the TCP 3-way handshake, t_{http2} is the HTTP/2 SETTINGS frame negotiation, t_{rpc} is the actual payload serialization/transmission, and $t_{teardown}$ is the TCP teardown (FIN/ACK). Because t_{tcp} and t_{http2} require multiple network round-trip times (RTTs) and significant CPU context switching, $S_{cpr} \gg t_{rpc}$.

In decentralized cloud-native architectures, the Client-side Proxy Routing (CPR) model dictates that the primary application container natively executes its own network routing and load balancing (e.g., utilizing native gRPC load balancing). This proxyless architecture minimizes intermediate network hops, optimizing baseline latency and reducing steady-state resource consumption. However, previous studies on proxyless service meshes demonstrate a critical vulnerability: the CPR model tightly couples the computational domain with the networking domain.

From a queueing theory perspective, the CPR model operates as a rigid $M/M/c$ queue. Under acute traffic surges, the network routing logic aggressively competes with the core application logic for limited CPU cycles and thread pools. When the system arrival rate approaches or exceeds the mathematical processing capacity ($\lambda \geq \lambda_{sat}$), the application's internal queue saturates rapidly, inevitably leading to dropped connections, unbounded tail latency, and catastrophic node failure.

B. PARADIGM 2: PERSISTENT CHANNEL REUSE (PCR)

In the optimized Persistent Channel Reuse (PCR) pattern, as shown in the PCR sections of Fig. 1 for both the synchronous and asynchronous gateway models, the architecture initializes a single, thread-safe gRPC channel during application startup. All incoming requests are multiplexed over this channel. The setup costs are amortized over millions of requests, approaching zero per request. Thus, the service time S_{pcr} simplifies to:

$$S_{pcr} = t_{rpc} \quad (6)$$

The PCR model completely decouples the data plane from the application by offloading routing, load balancing, and telemetry to a dedicated, out-of-process sidecar proxy (Envoy). This structural shift is dynamically enacted by the autonomic controller when saturation is detected.

While conventional research often criticizes the PCR model for introducing a continuous “service mesh tax”, permanent CPU, memory, and latency overheads during normal operations [19], [20], its architectural strength lies in robust queue management [21]. In the PCR model, the

injected sidecar acts as a highly optimized, asynchronous buffer [22]. It absorbs massive traffic spikes, manages connection backlogs, and applies intelligent backpressure, thereby protecting the underlying application container (whether running synchronous threads or an asynchronous event loop) from localized saturation.

C. QUEUEING THEORY AND NON-LINEAR SATURATION

The catastrophic latency spikes observed in empirical testing cannot be explained by relying only on S_{cpr} value. We apply an $M/M/c$ queueing model, where incoming requests arrive at rate λ and are serviced by c gateway computational resources at rate $\mu = 1/S_{gateway}$.

The utilization of the gateway system is $\rho = \frac{\lambda}{c \cdot \mu}$. According to Little's Law [23] and standard queueing theory, the average queueing delay W_{queue} grows asymptotically as utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho} \quad (7)$$

Because S_{cpr} is significantly larger than S_{pcr} , the service rate μ_{cpr} is artificially low. Therefore, under the CPR paradigm, the system reaches critical utilization ($\rho \rightarrow 1$) at a much lower arrival rate λ_{sat} . When λ exceeds this threshold, W_{queue} approaches infinity, resulting in the massive latency spikes and connection timeouts observed in our baseline data.

To fully contextualize the performance disparity between the CPR and PCR approaches, it is necessary to distinguish the roles of the TCP session, the HTTP/2 stream, and application-layer channel management. A TCP session represents the underlying Layer-4 network socket, which incurs significant latency overhead to establish. An HTTP/2 stream is a lightweight, logical sequence of frames operating within that single TCP session; this is the native mechanism gRPC uses to multiplex thousands of concurrent requests over one connection.

The fundamental issue of the CPR anti-pattern is that it actively avoids this multiplexing capability. By creating and tearing down a gRPC channel within the local request handler, CPR drives the underlying HTTP/2 library to establish and destroy a TCP session for each individual request. This limits the multiplexing capability to just one stream per connection. Contrarily, the PCR approach, enabled by the Envoy proxy, does not rebuild HTTP/2 features in the application layer. It is a structural design pattern that initializes a single, global TCP session, keeping the L4 connection persistent so that the native L7 HTTP/2 stream multiplexing functions as intended, and seamlessly absorbing the load that causes the CPR model to fail.

D. FRAMEWORK-SPECIFIC QUEUEING DYNAMICS

While the general queueing delay W_{queue} reflects system-level saturation, the structural behavior under high concurrency diverges fundamentally depending on the application framework's concurrency model. We distinguish between the

multi-threaded synchronous model and the single-threaded asynchronous event-loop model.

1) Synchronous Thread-Pool Model ($M/M/c$)

In the synchronous architecture (e.g., Flask [17]), the system relies on a thread pool of c parallel worker processes. Each worker handles a single connection sequentially. When a request initiates an un-multiplexed gRPC call under the CPR paradigm, the worker thread is blocked for the entire duration of S_{cpr} :

$$T_{\text{blocked}} = t_{\text{tcp}} + t_{\text{http2}} + t_{\text{rpc}} + t_{\text{teardown}} \quad (8)$$

When the arrival rate exceeds the aggregate thread processing capacity ($\lambda \geq c\mu_{cpr}$), all c workers become simultaneously occupied in I/O waiting states. Subsequent requests are diverted to the OS-level TCP listen backlog. Once this backlog fills, incoming SYN packets are dropped, driving $W_{\text{queue}} \rightarrow \infty$ and causing immediate client-side connection timeouts.

2) Asynchronous Event-Loop Model ($M/G/1 - PS$)

In contrast, the asynchronous architecture (e.g., FastAPI [18]) executes over an event loop using non-blocking I/O primitives. Rather than modeling the system as c discrete servers, it is more accurately characterized as an $M/G/1$ Processor Sharing (PS) queue, where a single execution thread distributes CPU time across N_{active} concurrent tasks [24].

When an asynchronous request initiates a gRPC call, it registers an I/O callback and yields control back to the event loop during the latency window $t_{\text{tcp}} + t_{\text{http2}}$. The service time distribution G is general rather than exponential due to asynchronous context switching. The average queueing delay under Processor Sharing is expressed as:

$$W_{\text{queue}, PS} = \frac{\rho}{1 - \rho} \cdot S_g \quad (9)$$

Because the event loop does not block physical threads during network negotiation, the effective capacity limit is decoupled from OS thread count and is instead constrained by event-loop tick duration and memory overhead. This enables the asynchronous gateway to hold thousands of partially completed connections in a pending state, delaying catastrophic packet drops far beyond the limits of the synchronous $M/M/c$ model.

The Algorithm 1 is represented to accommodate mathematical processes in this section. There are four main stages in the Algorithm 1: fetch real-time telemetry, initialization, analysis, and action stages. The fetch real-time telemetry stages will continuously accept all metrics from Prometheus required in the following process. Further acquired metric descriptions will be explained in Section IV. The initialization stages will identify which gateway framework stacks (Flask or FastAPI) the requester (frontend) uses to load the request. In the analysis stages, all metrics

Algorithm 1 Autonomic Controller with Math-based Predictive & SLA-Aware Dynamic Cutover

```

1: Inputs: Poll interval  $t_{\text{poll}}$ , Minimum traffic threshold
    $\lambda_{\text{min}} = 10$ , Latency SLA  $L_{\text{SLA}} = 50$  ms
2: Initialize:  $\text{cutover\_triggered} \leftarrow \text{False}$ 
3: while true do
4:   Sleep for  $t_{\text{poll}}$  seconds
5:   if  $\text{cutover\_triggered} = \text{True}$  then
6:     continue {Idling post-actuation}
7:   end if
8:   Fetch Real-time Telemetry
9:    $\lambda \leftarrow \text{QueryPrometheus}(\text{Rate of Requests (RPS)})$ 
10:   $S_{\text{cpr}} \leftarrow \text{QueryPrometheus}(\text{Average Service Time (s)})$ 
11:   $L_{p95} \leftarrow \text{QueryPrometheus}(\text{App P95 Latency (ms)})$ 
12:   $n_{\text{threads}} \leftarrow \text{QueryPrometheus}(\text{Active Threads})$ 
13:  if  $\lambda < \lambda_{\text{min}}$  or Telemetry is Null then
14:    continue {Wait for stable load}
15:  end if
16:  Framework Auto-Detection & Recalibration
17:  if  $n_{\text{threads}} > 50$  then
18:     $\text{mode} \leftarrow \text{SYNC (Flask)}$ 
19:  else
20:     $\text{mode} \leftarrow \text{ASYNC (FastAPI)}$ 
21:  end if
22:   $c \leftarrow 1.0$  {Normalized parallel compute capacity}
23:   $\tau_{\text{ratio}} \leftarrow 0.80$  {Target utilization threshold}
24:  Calculate Queueing Saturation Threshold
25:   $\lambda_{\text{sat}} \leftarrow \frac{c}{S_{\text{cpr}}}$ 
26:   $\lambda_{\text{trigger}} \leftarrow \lambda_{\text{sat}} \times \tau_{\text{ratio}}$ 
27:  Dual-Condition Evaluation
28:  if  $\lambda \geq \lambda_{\text{trigger}}$  and  $L_{p95} \geq L_{\text{SLA}}$  then
29:     $t_{\text{start}} \leftarrow \text{CurrentTime}()$ 
30:    PatchDeployment(sidecar.istio.io/inject="true")
31:    WaitUntilReady(Deployment Replicas)
32:     $\Delta t_{\text{rollout}} \leftarrow \text{CurrentTime}() - t_{\text{start}}$ 
33:    LogActuationMetrics( $\Delta t_{\text{rollout}}$ )
34:     $\text{cutover\_triggered} \leftarrow \text{True}$ 
35:  end if
36: end while

```

are evaluated using the mathematical model as discussed in this section. The results of the analysis will trigger specific steps to switch the patch, determining whether the action will be activated or not on the envoy sidecar deployment for the frontend. If the patch is considered to be deployed, this process involves evaluating the deployment time, which calculates how much time is spent on the envoy sidecar patch implementation, ensuring that the deployment responds appropriately based on the analysis outcomes.

IV. AUTONOMIC CONTROLLER ARCHITECTURE

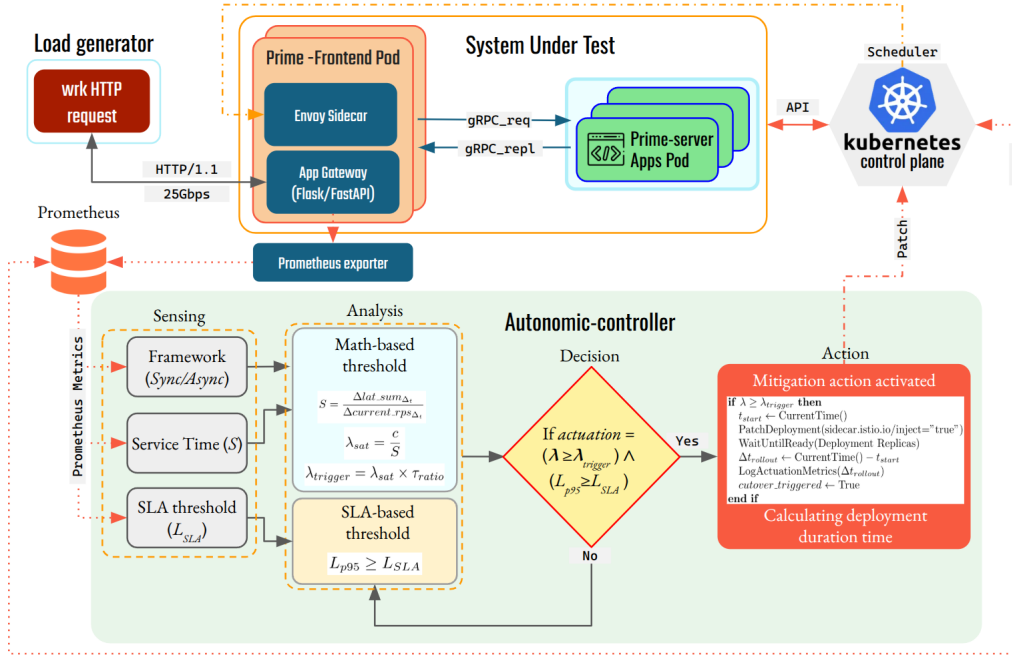


FIGURE 2. The autonomic controller architectures.

A. SENSING MECHANISM AND TELEMETRY ACQUISITION

The average service time, represented as S_{cpr} , serves as the fundamental calculation component in establishing the mathematical saturation limit of the autonomic controller. The controller relies on two complementary Prometheus telemetry metrics to accurately calculate the S_{cpr} variable in a distributed environment (e.g., RKE cluster). The controller continuously evaluates both metrics for further analysis: $grpc_client_roundtrip_latency_seconds_sum$ ($gRPC_{sum}$) and $grpc_client_roundtrip_latency_seconds_count$ ($gRPC_{count}$).

On the one hand, the $gRPC_{sum}$ metric serves as a continuous accumulator, gradually increasing over time. It tracks the absolute total seconds spent processing and responding to incoming remote procedure calls within the application. On the other hand, the $gRPC_{count}$ metric acts as a straightforward digital counter, registering the exact volume of discrete requests that have successfully completed their lifecycle.

To derive the real-time average service time for the control plane, the autonomic controller calculates the quotient of the rates of change of these two metrics over a sliding observation window. Mathematically, this relationship is expressed as the ratio of their respective variations over a specific time delta Δt :

$$S_{cpr} = \frac{\Delta gRPC_{sum} \Delta t}{\Delta gRPC_{count} \Delta t} \quad (10)$$

In the field of classical queuing theory, service time (S) is commonly used to represent the duration a physical or virtual server is actively engaged in a single unit of work. By utilizing native gRPC roundtrip metrics, the controller captures highly accurate, comprehensive processing measurements instead of

isolated internal application timers. This duration inherently includes data serialization, framework event-loop overhead, and the core CPU execution required for the prime number calculations. These processes are illustrated in the Fig. 2.

B. CALCULATION AND ANALYSIS

1) Predictive Mathematical Formulation

To mathematically formalize the system's performance under increasing target rates, the system architecture is modeled as an $M/M/c$ queue. This assumes a Poisson arrival process for incoming requests with rate λ (target rate), exponentially distributed service times with rate μ (where $\mu = 1/S_{cpr}$), and c parallel computing resources [25] [26]. For the system to remain stable, utilization must remain below capacity ($\rho < 1$). As the target rate λ increases toward the total service capacity $c\mu$, the system enters a state of high utilization. According to Erlang's C formula [27], the probability that an arriving request must wait in the queue (P_Q) is:

$$P_Q = \frac{\frac{(c\rho)^c}{c!(1-\rho)}}{\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)}} \quad (11)$$

Consequently, the average expected wait time in the queue (W_{queue}) grows non-linearly:

$$W_{queue} = \frac{P_Q}{c\mu(1-\rho)} \quad (12)$$

As $\rho \rightarrow 1$, W_{queue} approaches infinity. This mathematical phenomenon explains the severe spikes observed in empirical p99 tail latencies. Rather than waiting for system utilization to completely fail ($\rho \geq 1.0$), the controller continuously polls

real-time telemetry to compute the empirical service time (S_{cpr}) and calculates the mathematical saturation limit (λ_{sat}) based on the application's theoretical concurrency (c):

$$\lambda_{sat} = \frac{c}{S_{cpr}} \quad (13)$$

To guarantee survivability, the controller defines a predictive intervention boundary ($\lambda_{trigger}$) using a configurable safety threshold ratio (T_{ratio}):

$$\lambda_{trigger} = \lambda_{sat} \times T_{ratio} \quad (14)$$

2) SLA Evaluation and Bottleneck Verification

While the predictive trigger anticipates resource exhaustion using theoretical capacity as stated in (14), we also include a critical empirical verification of actual system degradation to avoid any impulsive actuation. The controller achieves this through real-time Service Level Agreement (SLA) monitoring, specifically targeting the 95th percentile application latency (L_{p95}). To acquire this metric accurately, the control plane queries the `grpc_client_roundtrip_latency_seconds_bucket` histogram exposed directly by the application runtime. By calculating the 95th quantile from the rate of change across these distribution buckets over a one-minute sliding window, the system extracts a stabilized, real-time measurement of the tail latency. We rely on this internal gRPC telemetry rather than the external HTTP network timers. It strictly separates the application's true compute and event-loop processing delays from external network issues, such as TCP backlog saturation or service queuing in the service mesh.

Targeting the 95th percentile ensures the controller focuses on the degraded experience of the slowest requests while filtering out extreme, transient outliers (like the 99th percentile) that are often caused by routine garbage collection or minor operating system context switching. This observed L_{p95} value is continuously evaluated against a rigid, configurable latency boundary (L_{SLA} , set to 35 ms). Within the hybrid controller mode, this SLA evaluation acts as the definitive proof of exhaustion. The mathematical capacity limit ($\lambda_{trigger}$) flags that the system should be saturated based on average service times, but the L_{p95} breach provides the empirical confirmation that the synchronous thread pool or asynchronous event loop has actually failed to process the current concurrent workload. By requiring this latency condition, the controller completely avoids false-positive mitigations during brief, harmless traffic bursts.

C. ACTUATION AND DYNAMIC CUTOVER

The control framework directly solves architectural inelasticity by bridging the CPR and PCR states dynamically. To prevent false positives caused by transient network jitter, the controller implements a strict dual-condition actuation gate. The dynamic cutover is only authorized if the predictive mathematical limit is breached *and* the empirical SLA is

actively failing (exceeding the 35 ms threshold). The final condition is defined as:

$$\text{Trigger} \rightarrow \text{True} \quad \text{if} \quad (\lambda \geq \lambda_{trigger}) \wedge (L_{p95} > 35 \text{ ms}) \quad (15)$$

Upon satisfying this dual-condition logic, the autonomic controller intervenes proactively. It initiates a dynamic patch to the frontend deployment within the Kubernetes control plane, applying the `sidecar.istio.io/inject="true"` annotation. This action forces a rolling restart, attaching the Envoy proxy to the workload in real-time. By dynamically shifting from a low-overhead CPR topology to a highly resilient PCR topology exclusively when these two thresholds intersect, the controller caps the connection bottleneck and neutralizes exponential queuing delay before catastrophic SLA violations occur.

V. EXPERIMENTAL SETUP AND EVALUATION SCENARIO

To validate the proposed mathematical model and pinpoint accurately where latency overhead originates, we conducted controlled experiments in a containerized distributed setting. Our evaluation follows a strict component isolation strategy: we first measure end-to-end performance across the full system and then benchmark the backend service alone in order to distinguish computation time from communication-related delays. This was achieved by deploying a stateless, deterministic backend and enforcing Kubernetes Guaranteed QoS (matching CPU/memory requests to limits) to eliminate resource throttling and "noisy neighbor" interference.

Our testbed uses a Python WSGI stacks (Flask/Gunicorn [17] and FastAPI/Uvicorn [18]) as a standard API gateway. We aim to check how gRPC connection churn impacts performance, not to compare frameworks. Synchronous threads are sensitive to blocking network inputs, highlighting queueing issues. Detailed infrastructure specifications are presented in Table 1.

TABLE 1. Infrastructure specification

Components	Load generator (A)	SUT (B)
CPU	Intel Xeon E-2274G (4 cores)	AMD EPYC 9655P (96 cores)
Memory	32GB	194GB
Storage	SSD 1TB	SSD 1TB
Network	Mellanox MT27800 25Gb	Mellanox MT27710 25Gb
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
K8s Cluster	RKE2 Control plane	RKE2 Worker

A. EXPERIMENTAL ENVIRONMENT AND ARCHITECTURE

The experiment was conducted on a Kubernetes cluster composed of multiple nodes, as shown in the Fig. 3. In the Kubernetes cluster, machine A serves as the control plane, which is tainted as a NoSchedule and NoExecute node. The end-to-end load request will be executed from this node. To completely isolate the deployment application, machine B serves as the system under test (SUT) that consists of two main microservice components:

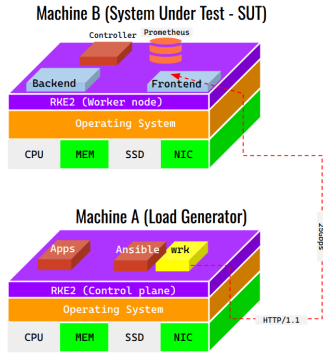


FIGURE 3. Kubernetes cluster environment.

- **gRPC Backend Service (prime-server):** A high-performance Python application responsible for CPU-intensive prime number verification. To eliminate backend capacity as a confounding variable, this component was over-provisioned with 10 identical replica pods, deployed behind a Kubernetes NodePort Service.
- **API Gateway (prime-frontend):** A Python Flask and FastAPI application running within Gunicorn and Uvicorn workers. This component acts as an HTTP/REST to gRPC proxy. It was strictly limited to 2 replica pods to intentionally force saturation and observe gateway-tier queuing dynamics under heavy load.

B. COMPONENT ISOLATION AND EVALUATION SCENARIO

To precisely calculate the application-layer gateway overhead, we utilized two specialized, open-source load-testing tools (*wrk2* [28] and *ghz* [29]), as shown in the Fig. 4. Both tools executed 30 iterations for each target throughput level, ranging from 200 to 1200 Requests Per Second (RPS).

1) End-to-end Comprehensive Evaluation

To evaluate the overall system performance and comprehensive user experience of the deployed microservices architecture, an end-to-end (E2E) evaluation stage was conducted. This scenario grabs the entire round-trip request lifecycle, effectively capturing the client's perception of performance as the traffic flows through the external network. Using *wrk2* as a load generator for a highly efficient HTTP benchmarking tool, we conduct the penetration test of the cluster via the frontend ingress, translate from HTTP to gRPC, process the request at the backend, and return the response to the client. This evaluation scenario is shown in the Fig. 4A.

The frontend serves as an API gateway, handling standard HTTP requests, transform the payload into gRPC protocol buffers, and routing the traffic to the backend prime-server-service. This topology ensures that the evaluation captures all systemic overhead within a 120-second time frame in 30 runs for each payload, including HTTP/gRPC translation costs, cluster network mesh latency, and the total resource usage of both frontend and backend components.

In this scenario, we also evaluate the system's adaptability, the framework efficiency, and the control logic. To obtain the E2E evaluation result, we incorporate interchangeable frontend implementations and multiple operational states. The API gateway tier was interchangeably evaluated using Flask (representing a traditional synchronous Web Server Gateway Interface-WSGI [30] handling profile) and FastAPI (representing a modern asynchronous Asynchronous Server Gateway Interface-ASGI [31] handling profile) to measure HTTP-to-gRPC translation overhead. Evaluations were strictly segmented between a Baseline state (relying entirely on default Kubernetes static scheduling) and a Controller-enabled state (where the custom dynamic control loop actively managed the system). During both Baseline state and Controller-enabled evaluations, the system was independently tested in CPR and PCR modes to determine their respective impacts on latency mitigation, resource scaling, and total end-to-end throughput under varying load intensities.

To maintain rigorous evaluation control and eliminate state leakage between load phases, each evaluation cycle executed an automated pipeline:

- **System Provisioning:** First, we ensure that all CPUs on the SUT node are in high-performance mode and remain in that state for further processes. Both the frontend and backend Kubernetes deployments and their associated services were dynamically applied. The script paused execution until all pods across both tiers achieved a Ready state.
- **Endpoint Resolution:** The external IP or NodePort of the Frontend service was dynamically resolved to establish the absolute target URL for the *wrk* client.
- **Comprehensive Telemetry Sampling:** A background daemon initiated continuous polling of container-level resource consumption using the Kubernetes Metrics API. This process simultaneously recorded CPU utilization (m) and memory usage (Mi) for frontend pods at 2-second intervals. This allowed us to conduct a comparative analysis of resource bottlenecks across the service mesh, whenever the current system is running with or without the controller.
- **Traffic Injection:** The *wrk* benchmarking tool injected sustained HTTP payloads for the designated duration (120 seconds) at the specified rate and concurrency.
- **Data Processing and Metric Extraction:** Upon completion of the injection phase, the script parsed the *wrk* outputs to extract the true client-facing metrics, include actual RPS (R_{act}), total Round-Trip time (T_{RTT}), latency distribution ($p50, p99$), and resources utilization (CPU and Memory).
- **Environment Teardown:** All deployments and services were completely decommissioned. The script enforced a mandatory cooldown period, monitoring the namespace until all resources were fully terminated before initiating the subsequent test iteration.

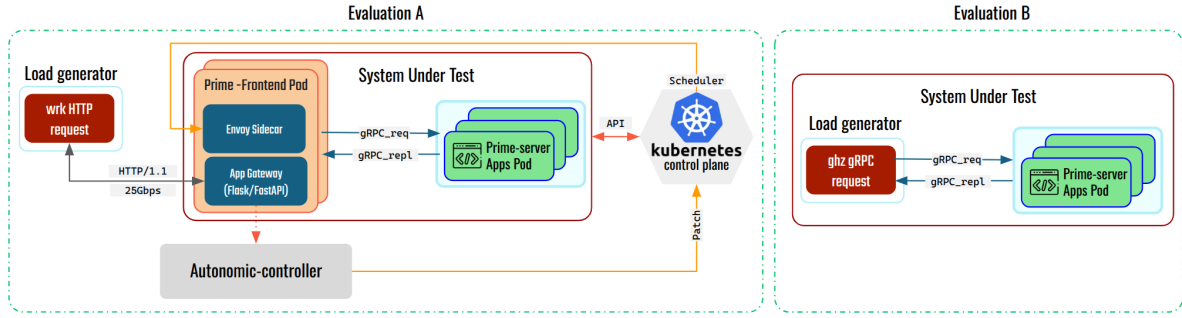


FIGURE 4. Evaluation scenario: (A) Comprehensive evaluations (end-to-end user experience); (B) Isolated backend service evaluation (compute time).

2) Backend service evaluation

Unlike end-to-end scenarios that evaluate the system as a whole, backend service evaluation aims to demonstrate that the backend service’s computational logic effectively handles gRPC requests from the load generator via cluster-level routing and service resolution. This evaluation scenario captures the total client-perceived performance when gRPC requests traverse the backend system path—from internal load generation (*ghz*—a gRPC performance tool) hosts that act as frontend service requests, penetrating directly through the backend service cluster. The backend service evaluation scenario is shown in Fig. 4B.

VI. RESULTS AND PERFORMANCE ANALYSIS

The empirical results have successfully proven that the predicted Asymptotic latency degradation is caused by the $M/M/c$ queuing model in the Connection-Per-Request (CPR) paradigm and have clearly demonstrated the transformational impact of Persistent Channel Reuse (PCR).

A. BASELINE: BACKEND GRPC PERFORMANCE

The *ghz* test results established an exceptional and highly stable baseline. Across the entire load spectrum (100 to 1000 RPS), the 10 prime-server backend pods consistently delivered a median (p50) latency of ~ 2.5 ms to 3.6 ms, with p90 latencies remaining firmly under 10 ms.

This flat latency curve definitively proved that the backend architecture was not the system bottleneck, as shown in the Fig. ???. By having 10 backend pods, we mathematically guaranteed that the backend possessed vastly more computational capacity than the frontend could ever saturate. The computational capacity of the prime number verification logic was highly resilient, isolating the source of any overarching performance degradation to the gateway tier.

Concurrency Configuration and Multiplexing: During empirical testing, both *ghz* and *wrk2* were configured to simulate high concurrency using a fixed pool of 50 simultaneous client connections across all target throughput levels (λ). It is critical to note how these concurrent accesses are handled internally by the optimized gateway. Under the PCR paradigm, the concurrent requests arriving at the gateway are not queued sequentially over the network; instead, the underlying gRPC framework maps these

concurrent application-level accesses into discrete, parallel HTTP/2 streams over a *single* shared TCP connection. This mechanism prevents connection exhaustion and directly validates our mathematical assumption that S_{pcr} approaches pure RPC execution time without queuing at the network socket layer.

B. UNOPTIMIZED SYSTEM: THE CPR BOTTLENECK

Testing the end-to-end system utilizing the CPR paradigm revealed a catastrophic bottleneck, as shown in the Fig. ??-A (upper side). At a moderate load of 400 RPS, the system maintained a median latency of ~ 400 ms. However, as throughput increased to 600 RPS, the system hit the critical utilization threshold ($\rho \rightarrow 1$).

Median latency exploded to $> 7,100$ ms, and p99 latency spiked past 14,000 ms. This exponential degradation perfectly aligns with the queueing theory model where service time (S_{pcr}) is artificially inflated by excessive TCP/HTTP2 handshake overheads for every transaction. The gateway workers were entirely occupied managing network state rather than processing business logic.

C. OPTIMIZED SYSTEM: PCR IMPLEMENTATION

By deploying the algorithmic optimization detailed in Section ?? (establishing a persistent, multiplexed gRPC channel at application startup - Algorithm ??), the performance profile of the system was qualitatively transformed.

At 600 RPS, the optimized system recorded a median latency of ~ 150 ms, as shown in the Fig. ??-B (lower side). This represents a $\sim 97.9\%$ reduction in end-to-end latency compared to the CPR paradigm. Furthermore, the system successfully sustained loads up to 800 RPS before demonstrating signs of stress, effectively doubling the system’s maximum throughput capacity without provisioning any additional infrastructure resources.

D. QUANTIFYING THE INHERENT PROXY LATENCY GAP

By subtracting the baseline backend network latency ($T_{network}$) established via *ghz* from the full end-to-end system latency (T_{total}) measured via *wrk2*, we can isolate the underlying application-layer tax. We define this remaining overhead as the Inherent Proxy Latency Gap—the baseline

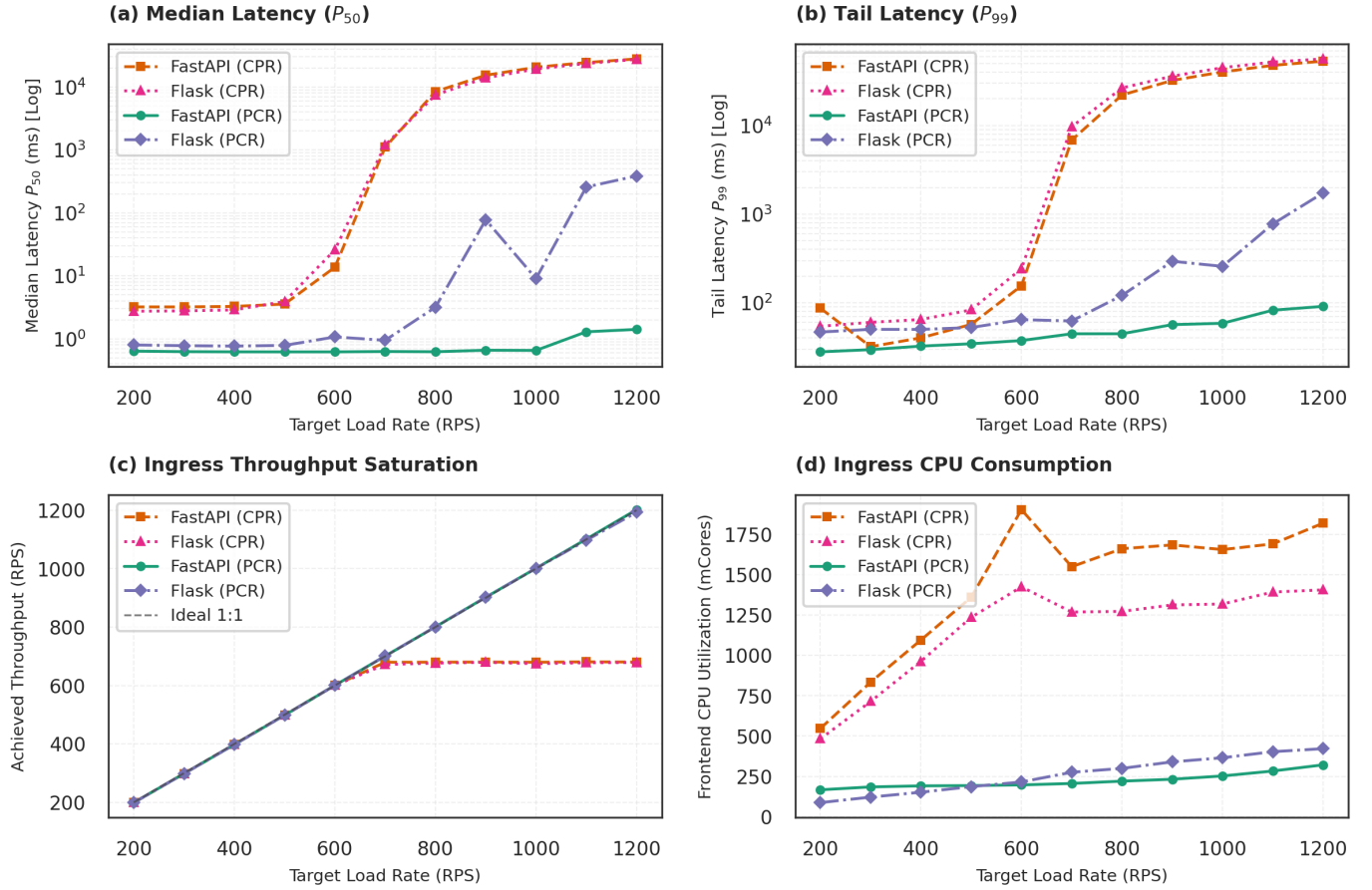


FIGURE 5. Baseline performance evaluation of frontend communication gateway (Synchronous and Asynchronous method) within Connection Per Request (CPR) and Persistent Connections Reuse (PCR) model: (a) median latency (p_{50}); (b) end-to-end latency (p_{99}); (c) systems throughput; (d) resource utilization (CPU).

latency strictly attributable to the framework's internal request handling, independent of network state management.

TABLE 2. Baseline performance comparison across connection management strategies (CPR and PCR) and architectural paradigms (synchronous - Flask and asynchronous - FastAPI).

Architecture	Load (RPS)	CPU (m)	Mem (Mi)	T _{comp} (ms)	T _{net} (ms)
Sync (Flask) CPR	200	482.1	147.1	0.84	4.98
Sync (Flask) CPR	600	1425.4	156.1	0.85	40.84
Sync (Flask) CPR	1000	1318.9	174.4	0.86	20454.8
Sync (Flask) PCR	200	86.9	140.7	0.83	2.77
Sync (Flask) PCR	600	215.1	141.1	0.87	6.83
Sync (Flask) PCR	1000	365.2	141.8	0.87	36.31
Async (FastAPI) CPR	200	546.4	346.1	0.83	5.55
Async (FastAPI) CPR	600	1904.2	355	0.82	25.14
Async (FastAPI) CPR	1000	1656.6	367.7	0.83	20518.2
Async (FastAPI) PCR	200	166.4	344.9	0.86	1.29
Async (FastAPI) PCR	600	196.7	345.4	0.84	3.02
Async (FastAPI) PCR	1000	252.4	345.5	0.86	5.71

As illustrated in Table ??, the results reveal a compelling paradox in the optimized PCR paradigm. Although the persistent channel implementation successfully averted queue saturation and achieved a 97% reduction in total latency

compared to the CPR baseline, a compositional analysis shows that the gateway tier still accounts for 97.5% of the remaining end-to-end response time.

This empirical gap directly connects our theoretical approach to the user experience. In our PCR model, the actual backend computational execution ($T_{network}$) requires only ~ 2.8 ms (2.5% of total time), while the remaining 107.2 ms is consumed by the gateway. Critically, this gap encompasses not only the Python WSGI [6] stack execution but also the protocol transcoding overhead—the computational cost of parsing incoming HTTP/1.1 JSON requests from *wrk2* and serializing them into HTTP/2 Protobuf messages for the gRPC backend.

This reveals a fundamental "Performance-Abstraction Paradox." While developers utilize API gateways to seamlessly bridge RESTful clients (HTTP/1.1) with high-performance microservices (gRPC), this protocol translation introduces a structural latency tax that dwarfs the business logic itself. Consequently, while our PCR optimization resolves protocol-level queue saturation, the absolute lower bound of system latency remains strictly dictated by the gateway's HTTP-to-gRPC transcoding efficiency.

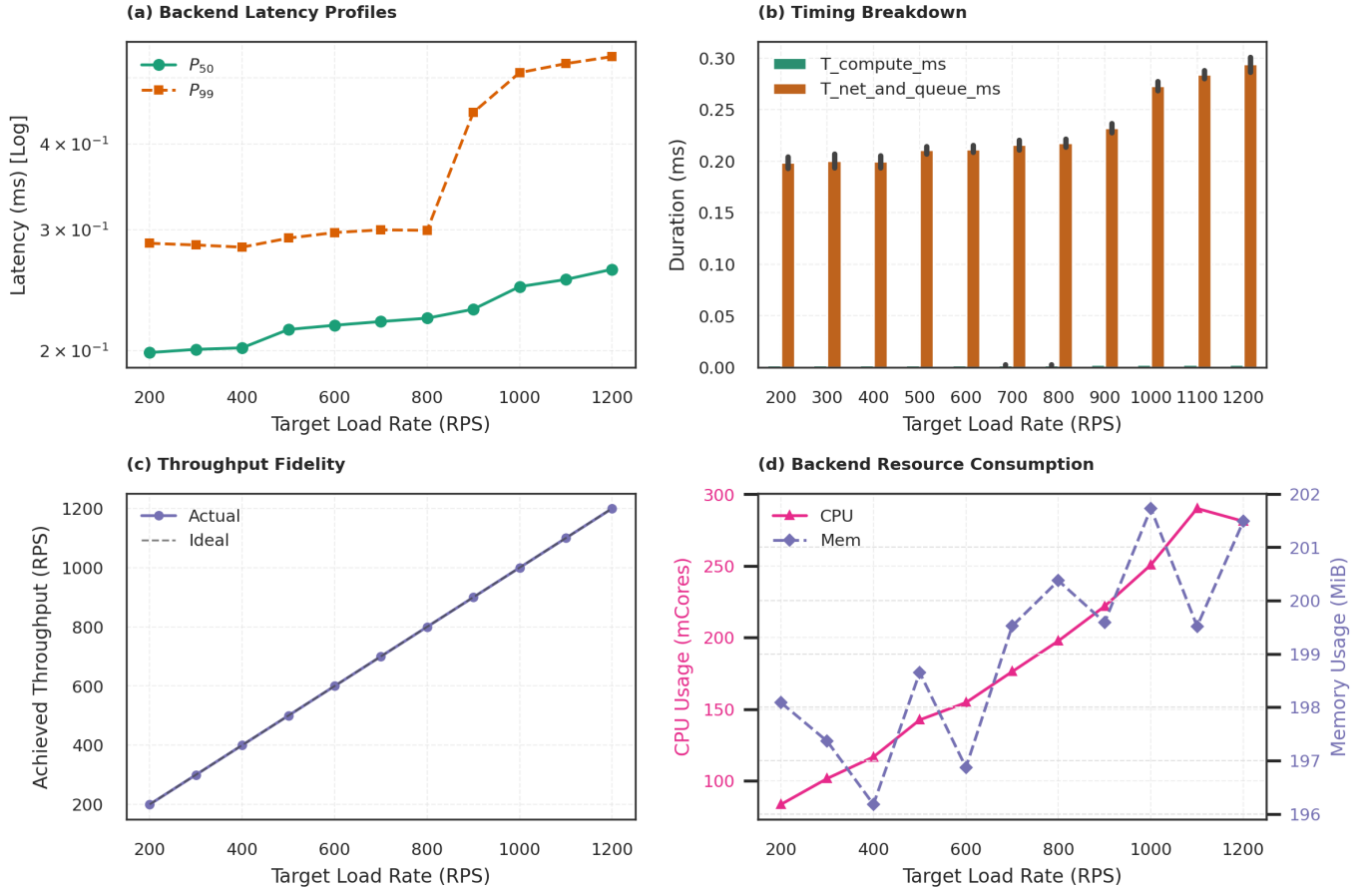


FIGURE 6. Baseline performance evaluation of backend service that evaluate directly to the backend server using ghz tool with gRPC load request: (a) median and tail latency (p_{50} , p_{99}); (b) compute vs net and queue time latency (t_{comp} , t_{net_queue}); (c) systems throughput; (d) resource utilization (CPU and Memory). These graphs are represent the result correspond to the evaluation scenario B on Fig. ??.

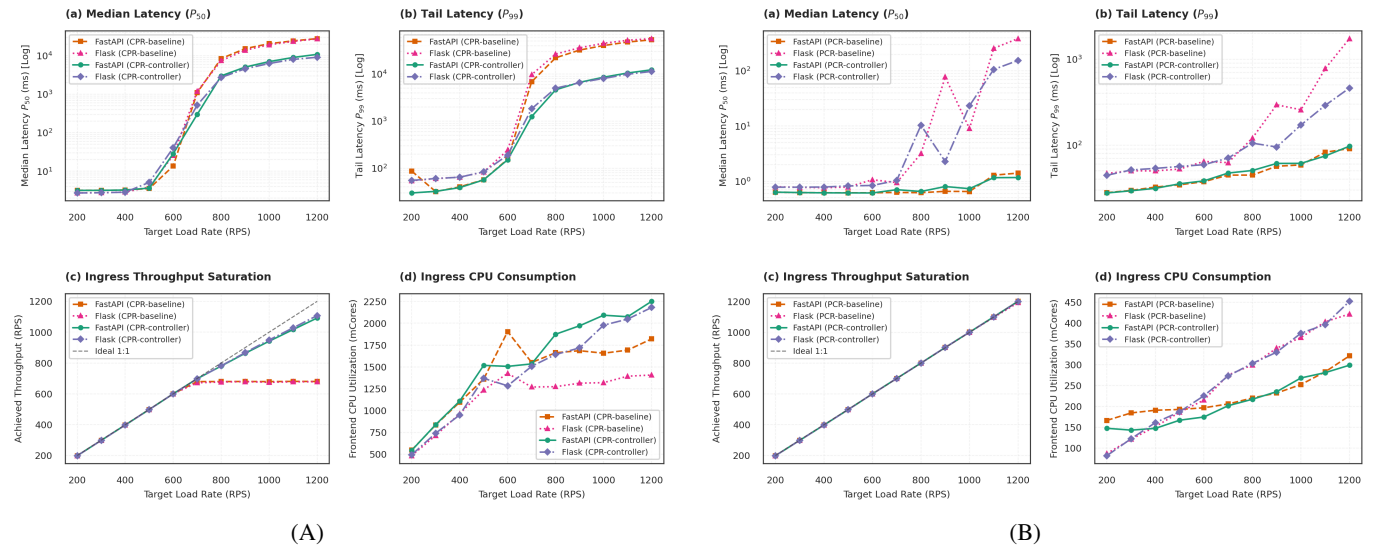


FIGURE 7. Baseline vs controller performance evaluation of frontend apps both in Synchronous and Asynchronous framework gateway within: (A) Connection Per-Request (CPR); (B) Persistent Connection Reuse (PCR) mode.

E. EMPIRICAL VALIDATION OF THE QUEUEING MODEL

To rigorously evaluate the mathematical framework proposed in Section ??, we mapped our empirical throughput and

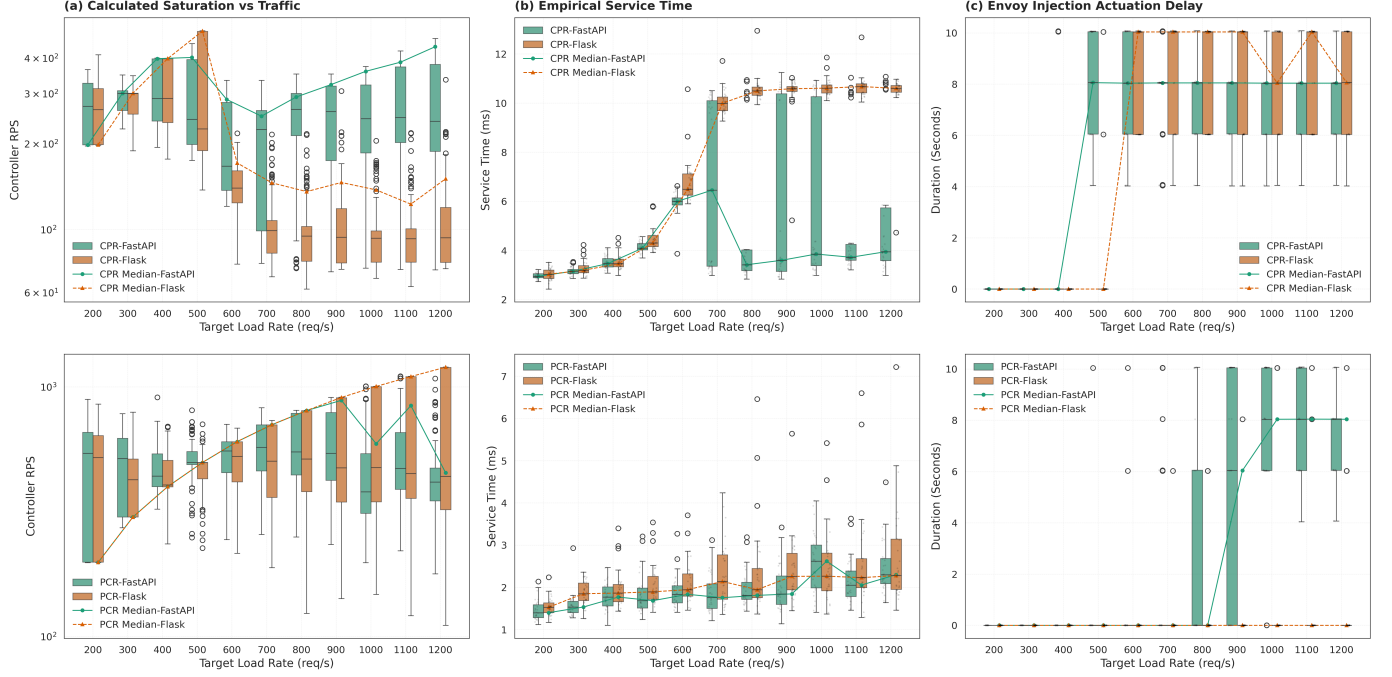


FIGURE 8. Autonomic controller performance evaluation in asynchronous (FastAPI/Uvicorn) and synchronous (Flask/Gunicorn) mode. Connection Per Request (CPR) - upper row, Persistent Connection Reuse (PCR) - lower row : (a) Calculated saturation point (λ_{sat}); (b) service time (S); (c) Actuation time when envoy deployed to frontend pods.

latency data to the $M/M/c$ queueing model. Our objective was to prove that the catastrophic latency spikes observed were not random system failures, but rather predictable queueing saturation events driven by connection churn.

In our system architecture, the gateway tier acts as the server pool (c workers). The arrival rate of incoming requests is defined as λ (varying from 100 to 1000 RPS). The service rate of the gateway, μ , is inversely proportional to the service time ($S_{gateway}$).

Under the Connection-Per-Request (CPR) paradigm, the service time (S_{cpr}) includes the mandatory TCP and HTTP/2 handshakes ($t_{tcp} + t_{http2}$). Because these network operations block gateway worker threads, μ_{cpr} is severely restricted.

Our empirical data aligns perfectly with the theoretical utilization formula:

$$\rho = \frac{\lambda}{c \cdot \mu}$$

According to Little's Law [23], queueing delay W_{queue} grows asymptotically as system utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho}$$

Evaluating the CPR Saturation Point: During our baseline CPR testing, the system maintained a median latency of ~ 400 ms at $\lambda = 400$ RPS, indicating high but stable utilization ($\rho < 1$). However, when the arrival rate increased to $\lambda = 600$ RPS, the median latency violently spiked to $> 7,100$ ms. This exponential degradation confirms that the low service rate (μ_{cpr}) caused the gateway to cross the critical threshold of $\rho = 1$ at a mere 600 RPS. At this

point, requests were arriving faster than connections could be negotiated, causing W_{queue} to theoretically approach infinity, which manifested empirically as 7-second tail latencies and connection timeouts.

Evaluating the PCR Optimization: By transitioning to the Persistent Channel Reuse (PCR) paradigm, we eliminated the handshake overhead, drastically reducing the service time to S_{pcr} and thereby significantly increasing the service rate (μ_{pcr}). Because $\mu_{pcr} \gg \mu_{cpr}$, the utilization ρ at the same arrival rate ($\lambda = 600$ RPS) remained well below 1.

To clarify the protocol dynamics, the Persistent Channel Reuse (PCR) approach does not reimplement HTTP/2 features; rather, it maintains a single global TCP session to allow native HTTP/2 stream multiplexing to function as intended. Conversely, the CPR anti-pattern actively sabotages this multiplexing capability by forcing the underlying library to establish and destroy a unique TCP session for every individual request.

Consequently, the queueing delay W_{queue} collapsed. The empirical result—a drop in latency from $> 7,100$ ms to exactly 150 ms at 600 RPS—serves as a direct physical validation of the $M/M/c$ model. It proves that correcting the $O(N)$ connection anti-pattern artificially scales the service rate μ , averting queue saturation without provisioning any additional hardware capacity (c).

VII. DISCUSSION

VIII. CONCLUSION AND FUTURE WORK

In this paper, we investigated the hidden performance costs of application-layer communication bottlenecks in

TABLE 3. Comparative End-to-End latency and controller suppression efficacy across load tiers (light - 200 RPS, moderate - 800 RPS, heavy - 1200 RPS).

Architecture	Load RPS	Baseline* RPS	Controller* RPS	RPS Suppres* (%)	Baseline p50 (ms)	Controller p50 (ms)	p50 Suppres (%)	Baseline p99 (ms)	Controller p99 (ms)	p99 Suppres (%)
Sync (Flask) CPR	200	199.1	199.1	0.0%	2.73	2.73	0.1%	54.35	55.2	-1.5%
Sync (Flask) CPR	800	676.6	781.8	15.6%	7452	2733.3	63.3%	26564.7	5004	81.2%
Sync (Flask) CPR	1200	678.6	1107.7	63.2%	27372.7	9129.2	66.6%	56520	11357.3	79.9%
Sync (Flask) PCR	200	199.1	199.1	0.0%	0.8	0.78	2.0%	46.8	44.6	4.8%
Sync (Flask) PCR	800	800.7	800.7	0.0%	3.8	10.3	-226.5%	121.02	105.4	12.9%
Sync (Flask) PCR	1200	1192.8	1200.2	0.6%	385.1	154.6	59.8%	1737.3	462.1	73.4%
Async (FastAPI) CPR	200	199.1	199.1	0.0%	3.2	3.2	0.3%	87.2	29.9	65.7%
Async (FastAPI) CPR	800	679.8	780.3	14.8%	8447	3005	64.4%	22125.3	4622	79.1%
Async (FastAPI) CPR	1200	680.3	1092.1	60.5%	27870	10856	61.0%	53320	12305	76.9%
Async (FastAPI) PCR	200	199.1	199.1	0.0%	0.6	0.6	0.1%	27.9	27.7	0.9%
Async (FastAPI) PCR	800	800.7	800.7	0.0%	0.6	0.6	-5.1%	44.7	50.6	-13.2%
Async (FastAPI) PCR	1200	1201.8	1201.8	0.0%	1.4	1.2	16.9%	91.2	97.2	-6.6%

* Systems throughput

TABLE 4. Autonomic controller telemetry, actuation metrics, and resource footprint.

Architecture	Target load Intervention Threshold	Degraded Capacity at Trigger λ_{sat} (RPS)	Projected Crash Point $\lambda_{trigger}$ (RPS)	Actuation/Cutover $\Delta T_{rollout}$ Time (s)	Controller CPU Overhead (m)	Controller Mem Overhead (Mi)
Sync (Flask) CPR	≥ 500 RPS	83.7	104.6	8.3	9.3	56.6
Sync (Flask) PCR	≥ 700 RPS	148.9	186.1	7.8	8.3	56.8
Async (FastAPI) CPR	≥ 600 RPS	168.3	210.4	8.27	9.1	6.6
Async (FastAPI) PCR	Never Triggered	N/A	N/A	0.00	8.9	56.8

TABLE 5. Summary of proposed autonomic-controller performance in handling the latency reduction, mitigation effectiveness and increasing the throughput systems under saturated load.

Architecture & Mode	Target Load (RPS)	Throughput Gain (%)	p50 Latency Reduction (%)	p99 Latency Reduction (%)	Peak p99 Mitigation
Sync (Flask) CPR	800	+15.6%	63.3%	81.2%	26.5s $\rightarrow$ 5.0s
Sync (Flask) CPR	1200	+63.2%	66.6%	79.9%	56.5s $\rightarrow$ 11.4s
Sync (Flask) PCR	1200	+0.6%	59.8%	73.4%	1.7s $\rightarrow$ 0.5s
Async (FastAPI) CPR	800	+14.8%	64.4%	79.1%	22.1s $\rightarrow$ 4.6s
Async (FastAPI) CPR	1200	+60.5%	61.0%	76.9%	53.3s $\rightarrow$ 12.3s

microservice architectures, focusing on the gRPC connection churn flaw within API gateways. We modeled the request patterns of the "connection-per-request" (CPR) anti-pattern and contrasted it with a refactored Persistent Channel Reuse (PCR) approach. The observation of actual runs demonstrated that refactoring the gateway implementation to the PCR approach yielded a 97% reduction in median system latency—dropping from over 7,100 ms to 150 ms under a peak load of 600 RPS. This study proves that correcting application-layer protocol mismanagement successfully moves the performance bottleneck away from the infrastructure gateway and to the compute backend, highlighting that refactoring the gateway code is a critical aspect to scalability as the underlying architectural design.

Future work will expand this steady-state benchmarking methodology into the domain of Chaos Engineering. By introducing controlled faults (e.g., node resource starvation, sudden pod termination) under a persistent load.

IX. REFERENCE EXAMPLES

• Basic format for books:

J. K. Author, "Title of chapter in the book," in *Title of His Published Book*, xth ed. City of Publisher, (only U.S.

State), Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx–xxx.

See [32], [33].

• Basic format for periodicals:

J. K. Author, "Name of paper," *Abbrev. Title of Periodical*, vol. x, no. x, pp. xxx–xxx, Abbrev. Month, year, DOI. 10.1109.XXX.123456.

See [34]–[36].

• Basic format for reports:

J. K. Author, "Title of report," *Abbrev. Name of Co.*, City of Co., Abbrev. State, Country, Rep. xxx, year.

See [37], [38].

• Basic format for handbooks:

Name of Manual/Handbook, x ed., *Abbrev. Name of Co.*, City of Co., Abbrev. State, Country, year, pp. xxx–xxx.

See [39], [40].

• Basic format for books (when available online):

J. K. Author, "Title of chapter in the book," in *Title of Published Book*, xth ed. City of Publisher, State, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx–xxx. [Online]. Available: <http://www.web.com>

See [41]–[44].

• Basic format for journals (when available online):

- J. K. Author, "Name of paper," *Abbrev. Title of Periodical*, vol. x, no. x, pp. xxx–xxx, Abbrev. Month, year. Accessed on: Month, Day, year, DOI: 10.1109.XXX.123456, [Online].
See [45]– [47].
- *Basic format for papers presented at conferences (when available online):*
J.K. Author. (year, month). Title. presented at abbrev. conference title. [Type of Medium]. Available: site/path/file
See [48].
 - *Basic format for reports and handbooks (when available online):*
J. K. Author. "Title of report," Company. City, State, Country. Rep. no., (optional: vol./issue), Date. [Online] Available: site/path/file
See [49], [50].
 - *Basic format for computer programs and electronic documents (when available online):*
Legislative body. Number of Congress, Session. (year, month day). *Number of bill or resolution, Title*. [Type of medium]. Available: site/path/file
See [51].
 - *Basic format for patents (when available online):*
Name of the invention, by inventor's name. (year, month day). Patent Number [Type of medium]. Available: site/path/file
See [52].
 - *Basic format for conference proceedings (published):*
J. K. Author, "Title of paper," in *Abbreviated Name of Conf.*, City of Conf., Abbrev. State (if given), Country, year, pp. xxxxxx.
See [53].
 - *Example for papers presented at conferences (unpublished):*
See [54].
 - *Basic format for patents:*
J. K. Author, "Title of patent," U.S. Patent x xxx xxx, Abbrev. Month, day, year.
See [55].
 - *Basic format for theses (M.S.) and dissertations (Ph.D.):*
 - 1) J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.
 - 2) J. K. Author, "Title of dissertation," Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.
See [56], [57].
 - *Basic format for the most common types of unpublished references:*
 - 1) J. K. Author, private communication, Abbrev. Month, year.
 - 2) J. K. Author, "Title of paper," unpublished.
 - 3) J. K. Author, "Title of paper," to be published.
See [58]– [60].
 - *Basic formats for standards:*

- 1) *Title of Standard*, Standard number, date.
- 2) *Title of Standard*, Standard number, Corporate author, location, date.

See [61], [62].

- *Article number in reference examples:*

See [63], [64].

- *Example when using et al.:*

See [65].

ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in American English is without an "e" after the "g." Use the singular heading even if you have many acknowledgments. Avoid expressions such as "One of us (S.B.A.) would like to thank . . ." Instead, write "F. A. Author thanks . . ." In most cases, sponsor and financial support acknowledgments are placed in the unnumbered footnote on the first page, not here.

REFERENCES

- [1] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, "Online testing of RESTful APIs: Promises and challenges," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Singapore, 2022, pp. 408–420, doi: 10.1145/3540250.3549144.
- [2] P. L. Ventre, M. M. Tajiki, S. Salsano, and C. Filsfils, "SDN architecture and southbound APIs for IPv6 segment routing enabled wide area networks," *IEEE Transaction on Network and Service Management*, vol. 15, pp. 1378–1392, 2018, doi: 10.1109/TNSM.2018.2876251.
- [3] X. Zhu, G. She, B. Xue, Y. Zhang, Y. Zhang, X. K. Zou, X. Duan, P. He, A. Krishnamurthy, and M. Lentz, "Dissecting overheads of service mesh sidecars," in *Proceedings of the 2023 ACM Symposium on Cloud Computin*, 2023, pp. 142–157, doi: 10.1145/3620678.3624652.
- [4] L. Guo and J. Y. B. Lee, "Stateful-TCP—a new approach to accelerate TCP slow-start," *IEEE Access*, vol. 8, pp. 195955–195970, 2020, doi: 10.1109/ACCESS.2020.3034129.
- [5] A. Sampson, Y. Saito, and R. Chan, "Bebop: A Branchless Data Interchange Format and RPC Protocol," *arXiv preprint arXiv:2604.09591*, Jun. 2026.
- [6] P. J. Eby, "PEP 333 – Python Web Server Gateway Interface v1.0," Sep. 27, 2010. [Online]. Available: <https://peps.python.org/pep-0333/>. [Accessed: Sept. 24, 2026].
- [7] T. Pinheiro, M. Mialaret, P. Pereira, L. Lins, D. Silva, J. Dantas, and P. Maciel, "Performance modeling of microservices with circuit breakers using stochastic Petri nets," in *2024 IEEE International Systems Conference (SysCon)*, 2024, pp. 1–8, doi: 10.1109/SysCon61195.2024.10553490.
- [8] L. Carnevali, M. Paolieri, R. Reali, L. Scommegna, and E. Vicario, "Compositional coordinated resource provisioning in workflows with stochastic durations," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 9, pp. 1937–1954, 2025, doi: 10.1109/TPDS.2025.3585821.
- [9] B. D'Auria, I. Adan, R. Bekker, and V. Kulkarni, "An M/M/c queue with queueing-time dependent service rates," *European Journal of Operational Research*, vol. 299, pp. 566–579, 2021, doi: 10.1016/j.ejor.2021.12.023.
- [10] M. A. Alzahrani and E. Y. Albassam, "A scalable microservice architecture for autonomous IoT systems in smart transportation: Integration of edge computing, real-time processing, and security," *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3691572.
- [11] M. B. Taha, Y. Sanjalawe, A. Al-Daraiseh, S. Fraihat, and S. R. Al-E'mari, "Proactive auto-scaling for service function chains in cloud computing based on deep learning," *IEEE Access*, vol. 12, pp. 38575–38593, 2024, doi: 10.1109/ACCESS.2024.3375772.
- [12] F. Biondani, F. Tosoni, N. Dall'Ora, E. Fraccaroli, D. S. Cheng, and F. Fummi, "MATE: A methodology for connecting automatic test equipment in Industry 4.0," *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3687714.

- [13] D. Armstrong, K. Djemame, and R. Kavanagh, "Towards energy aware cloud computing application construction," *Journal of Cloud Computing*, vol. 6, no. 1, p. 14, 2017, doi: 10.1186/s13677-017-0083-2.
- [14] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Machine learning-based scaling management for kubernetes edge clusters," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 958–972, 2021, doi: 10.1109/TNSM.2021.3052837.
- [15] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-Oriented microservices," in *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, pp. 805–825, 2020.
- [16] R. Dautov, I. Paraskakis, and M. Stannett, "Utilising stream reasoning techniques to underpin an autonomous framework for cloud application platforms," *Journal of Cloud Computing*, vol. 3, no. 1, p. 13, 2014, doi: 10.1186/s13677-014-0013-5.
- [17] Unicorn, "Unicorn: Serve Python on the Web." [Online]. Available: <https://unicorn.org/>. Accessed on: September 24, 2026].
- [18] Uvicorn, "An ASGI web server, for Python." [Online]. Available: <https://uvicorn.dev/>. Accessed on: September 24, 2026].
- [19] M. Chen, M. T. Islam, M. R. Read, and R. Buyya, "Trade: Network and traffic-aware adaptive scheduling for microservices under dynamics," *IEEE Transactions on Parallel and Distributed Systems*, vol. 37, no. 1, pp. 76–89, 2025, doi: 10.1109/TPDS.2025.3626424.
- [20] L. Zhang, Y. Yang, K. Pang, T. Chen, and X. Yue, "An adaptive encryption scheme for secure microservice communication via variable-length keys and obfuscation," *Journal of Cloud Computing*, vol. 15, no. 1, p. 80, 2026, doi: 10.1186/s13677-026-00899-1.
- [21] Z. Xiang, Y. Zheng, D. Wang, J. Taheri, Z. Zheng, and M. Guo, "Cost-effective and robust service provisioning in multi-access edge computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 10, pp. 1765–1779, 2024, doi: 10.1109/TPDS.2024.3435929.
- [22] G. L. Santos, P. T. Endo, M. F. F. S. L. Tigre, L. G. F. S. Silva, D. Sadok, J. Kelner, and T. Lynn, "Analyzing the availability and performance of an e-health system integrated with edge, fog and cloud infrastructures," *Journal of Cloud Computing*, vol. 7, no. 1, p. 16, 2018, doi: 10.1186/s13677-018-0118-3.
- [23] J. D. C. Little, "OR forum—Little's law as viewed on its 50th anniversary," *Oper. Res.*, vol. 59, no. 3, pp. 536–549, 2011, doi: 10.1287/opre.1110.0940.
- [24] D. Dalai, S. Babu, V. B. Sukumaran, and B. S. Manoj, "A novel space-based hosting approach for ultra low latency web services," *IEEE Access*, vol. 12, pp. 142838–142862, 2024, doi: 10.1109/ACCESS.2024.3462252.
- [25] H. C. Tijms, *A first course in stochastic models*, vol. 2, Wiley New York, 2003.
- [26] P. Banerjee, S. Roy, A. Sinha, M. Hassan, S. Burje, A. Agrawal, A. K. Bairagi, S. Alshathri, and W. El-shafai, "MTD-DHJS: Makespan-Optimized Task Scheduling Algorithm for Cloud Computing With Dynamic Computational Time Prediction," *IEEE Access*, vol. 11, pp. 105578–105618, 2023, doi: 10.1109/ACCESS.2023.3318553.
- [27] I. Angus et al., "An introduction to Erlang B and Erlang C," *Telemanagement*, vol. 187, pp. 6–8, 2001.
- [28] G. Tene, "A HTTP benchmarking tool based mostly on wrk," Jun. 28, 2016. [Online]. Available: <https://github.com/giltene/wrk2>. [Accessed: May 14, 2026].
- [29] B. Djurkovic, "gRPC benchmarking and load testing tool," Dec. 7, 2025. [Online]. Available: <https://github.com/bojand/ghz>. [Accessed: May 14, 2026].
- [30] WSGI Team, "What is WSGI," 2015. [Online]. Available: <https://wsgi.readthedocs.io/en/latest/what.html>. [Accessed: Sept. 24 2026].
- [31] ASGI Team, "ASGI Documentation," 2018. [Online]. Available: <https://asgi.readthedocs.io/en/latest/>. [Accessed: Sept. 24 2026].
- [32] G. O. Young, "Synthetic structure of industrial plastics," in *Plastics*, 2nd ed., vol. 3, J. Peters, Ed. New York, NY, USA: McGraw-Hill, 1964, pp. 15–64.
- [33] W.-K. Chen, *Linear Networks and Systems*. Belmont, CA, USA: Wadsworth, 1993, pp. 123–135.
- [34] J. U. Duncombe, "Infrared navigation—Part I: An assessment of feasibility," *IEEE Trans. Electron Devices*, vol. ED-11, no. 1, pp. 34–39, Jan. 1959, 10.1109/TED.2016.2628402.
- [35] E. P. Wigner, "Theory of traveling-wave optical laser," *Phys. Rev.*, vol. 134, pp. A635–A646, Dec. 1965.
- [36] E. H. Miller, "A note on reflector arrays," *IEEE Trans. Antennas Propagat.*, to be published.
- [37] E. E. Reber, R. L. Michell, and C. J. Carter, "Oxygen absorption in the earth's atmosphere," Aerospace Corp., Los Angeles, CA, USA, Tech. Rep. TR-0200 (4230-46)-3, Nov. 1988.
- [38] J. H. Davis and J. R. Cogdell, "Calibration program for the 16-foot antenna," Elect. Eng. Res. Lab., Univ. Texas, Austin, TX, USA, Tech. Memo. NGL-006-69-3, Nov. 15, 1987.
- [39] *Transmission Systems for Communications*, 3rd ed., Western Electric Co., Winston-Salem, NC, USA, 1985, pp. 44–60.
- [40] *Motorola Semiconductor Data Manual*, Motorola Semiconductor Products Inc., Phoenix, AZ, USA, 1989.
- [41] G. O. Young, "Synthetic structure of industrial plastics," in *Plastics*, vol. 3, Polymers of Hexadromicon, J. Peters, Ed., 2nd ed. New York, NY, USA: McGraw-Hill, 1964, pp. 15–64. [Online]. Available: <http://www.bookref.com>.
- [42] *The Founders' Constitution*, Philip B. Kurland and Ralph Lerner, eds., Chicago, IL, USA: Univ. Chicago Press, 1987. [Online]. Available: <http://press-pubs.uchicago.edu/founders/>
- [43] The Terahertz Wave eBook. ZOmega Terahertz Corp., 2014. [Online]. Available: http://dl.z-thz.com/eBook/zomegaebookpdf_1206_sr.pdf. Accessed on: May 19, 2014.
- [44] Philip B. Kurland and Ralph Lerner, eds., *The Founders' Constitution*. Chicago, IL, USA: Univ. of Chicago Press, 1987, Accessed on: Feb. 28, 2010, [Online] Available: <http://press-pubs.uchicago.edu/founders/>
- [45] J. S. Turner, "New directions in communications," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 1, pp. 11–23, Jan. 1995.
- [46] W. P. Risk, G. S. Kino, and H. J. Shaw, "Fiber-optic frequency shifter using a surface acoustic wave incident at an oblique angle," *Opt. Lett.*, vol. 11, no. 2, pp. 115–117, Feb. 1986.
- [47] P. Kopyt et al., "Electric properties of graphene-based conductive layers from DC up to terahertz range," *IEEE THz Sci. Technol.*, to be published. DOI: 10.1109/TTHZ.2016.2544142.
- [48] PROCESS Corporation, Boston, MA, USA. Intranets: Internet technologies deployed behind the firewall for corporate productivity. Presented at INET96 Annual Meeting. [Online]. Available: <http://home.process.com/Intranets/wp2.htm>
- [49] R. J. Hijmans and J. van Etten, "Raster: Geographic analysis and modeling with raster data," R Package Version 2.0-12, Jan. 12, 2012. [Online]. Available: <http://CRAN.R-project.org/package=raster>
- [50] Teralyzer. Lytera UG, Kirchhain, Germany [Online]. Available: http://www.lytera.de/Terahertz_THz_Spectroscopy.php?id=home, Accessed on: Jun. 5, 2014.
- [51] U.S. House. 102nd Congress, 1st Session. (1991, Jan. 11). *H. Con. Res. 1, Sense of the Congress on Approval of Military Action*. [Online]. Available: LEXIS Library: GENFED File: BILLS
- [52] Musical toothbrush with mirror, by L.M.R. Brooks. (1992, May 19). Patent D 326 189 [Online]. Available: NEXIS Library: LEXPAT File: DES
- [53] D. B. Payne and J. R. Stern, "Wavelength-switched passively coupled single-mode optical network," in *Proc. IOOC-ECOC*, Boston, MA, USA, 1985, pp. 585–590.
- [54] D. Ebehard and E. Voges, "Digital single sideband detection for interferometric sensors," presented at the 2nd Int. Conf. Optical Fiber Sensors, Stuttgart, Germany, Jan. 2–5, 1984.
- [55] G. Brandli and M. Dick, "Alternating current fed power supply," U.S. Patent 4 084 217, Nov. 4, 1978.
- [56] J. O. Williams, "Narrow-band analyzer," Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, USA, 1993.
- [57] N. Kawasaki, "Parametric study of thermal and chemical nonequilibrium nozzle flow," M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993.
- [58] A. Harrison, private communication, May 1995.
- [59] B. Smith, "An approach to graphs of linear forms," unpublished.
- [60] A. Brahms, "Representation error for real numbers in binary computer arithmetic," IEEE Computer Group Repository, Paper R-67-85.
- [61] IEEE Criteria for Class IE Electric Systems, IEEE Standard 308, 1969.
- [62] Letter Symbols for Quantities, ANSI Standard Y10.5-1968.
- [63] R. Fardel, M. Nagel, F. Nuesch, T. Lippert, and A. Wokaun, "Fabrication of organic light emitting diode pixels by laser-assisted forward transfer," *Appl. Phys. Lett.*, vol. 91, no. 6, Aug. 2007, Art. no. 061103.
- [64] J. Zhang and N. Tansu, "Optical gain and laser characteristics of InGaN quantum wells on ternary InGaIn substrates," *IEEE Photon. J.*, vol. 5, no. 2, Apr. 2013, Art. no. 2600111
- [65] S. Azodolmolky et al., Experimental demonstration of an impairment aware network planning and operation tool for transparent/translucent

optical networks,” *J. Lightw. Technol.*, vol. 29, no. 4, pp. 439–448, Sep. 2011.



FIRST A. AUTHOR received the B.S. and M.S. degrees in aerospace engineering from the University of Virginia, Charlottesville, in 2001 and the Ph.D. degree in mechanical engineering from Drexel University, Philadelphia, PA, in 2008.

From 2001 to 2004, he was a Research Assistant with the Princeton Plasma Physics Laboratory. Since 2009, he has been an Assistant Professor with the Mechanical Engineering Department, Texas A&M University, College Station. He is the

author of three books, more than 150 articles, and more than 70 inventions. His research interests include high-pressure and high-density nonthermal plasma discharge processes and applications, microscale plasma discharges, discharges in liquids, spectroscopic diagnostics, plasma propulsion, and innovation plasma applications. He is an Associate Editor of the journal *Earth, Moon, Planets*, and holds two patents.

Dr. Author was a recipient of the International Association of Geomagnetism and Aeronomy Young Scientist Award for Excellence in 2008, and the IEEE Electromagnetic Compatibility Society Best Symposium Paper Award in 2011.

THIRD C. AUTHOR, JR. (M'87) received the B.S. degree in mechanical engineering from National Chung Cheng University, Chiayi, Taiwan, in 2004 and the M.S. degree in mechanical engineering from National Tsing Hua University, Hsinchu, Taiwan, in 2006. He is currently pursuing the Ph.D. degree in mechanical engineering at Texas A&M University, College Station, TX, USA.

From 2008 to 2009, he was a Research Assistant with the Institute of Physics, Academia Sinica, Taipei, Taiwan. His research interest includes the development of surface processing and biological/medical treatment techniques using nonthermal atmospheric pressure plasmas, fundamental study of plasma sources, and fabrication of micro- or nanostructured surfaces.

Mr. Author's awards and honors include the Frew Fellowship (Australian Academy of Science), the I. I. Rabi Prize (APS), the European Frequency and Time Forum Award, the Carl Zeiss Research Award, the William F. Meggers Award and the Adolph Lomb Medal (OSA).

...



SECOND B. AUTHOR (M'76–SM'81–F'87) and all authors may include biographies. Biographies are often not included in conference-related papers. This author became a Member (M) of IEEE in 1976, a Senior Member (SM) in 1981, and a Fellow (F) in 1987. The first paragraph may contain a place and/or date of birth (list place, then date). Next, the author's educational background is listed. The degrees should be listed with type of degree in what field, which institution, city, state,

and country, and year the degree was earned. The author's major field of study should be lower-cased.

The second paragraph uses the pronoun of the person (he or she) and not the author's last name. It lists military and work experience, including summer and fellowship jobs. Job titles are capitalized. The current job must have a location; previous positions may be listed without one. Information concerning previous publications may be included. Try not to list more than three books or published articles. The format for listing publishers of a book within the biography is: title of book (publisher name, year) similar to a reference. Current and previous research interests end the paragraph.

The third paragraph begins with the author's title and last name (e.g., Dr. Smith, Prof. Jones, Mr. Kajor, Ms. Hunter). List any memberships in professional societies other than the IEEE. Finally, list any awards and work for IEEE committees and publications. If a photograph is provided, it should be of good quality, and professional-looking. Following are two examples of an author's biography.